ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS

Semestre 9 · Année universitaire 2026-2027

Rapport de projet

Deep Learning for Spare Parts Management

Pilotage conjoint de la maintenance et du stock de pièces de rechange d'une flotte de machines industrielles

Réalisé par Adam Elmadani

Encadré par Pr. Aziza Belmaati

Pr. Oumaima Bounou

Version 1.0, octobre 2026

Résumé

Ce projet étudie le pilotage conjoint de la maintenance et du stock de pièces de rechange d'une flotte de 100 machines réparties en deux familles, l'une vieillissant à l'usage, l'autre au calendrier. Un simulateur à événements discrets reproduit pannes, maintenances, approvisionnements, défauts de fabrication et télémétrie ; calibré sur le jeu Microsoft Azure Predictive Maintenance, il retrouve le nombre de pannes de chaque composant sur l'année 2015 à moins de 10 % près. Des réseaux de survie récurrents et convolutifs estiment le risque de panne de chaque composant à partir de 48 heures de signaux ; ils dépassent LightGBM sur données simulées et, à 7 jours, sur données réelles. Une politique prédictive fondée sur ces réseaux réduit le coût annuel de 2,1 % par rapport à une politique à seuils fixes optimisés, écart significatif sur 30 années simulées, mais une maintenance conditionnelle simple fait mieux encore, avec 9,6 % d'économie. Les agents d'apprentissage par renforcement profond (PPO avec masquage d'actions, DQN, Branching Dueling Q-Network) n'atteignent pas la politique à seuils optimisés : le meilleur, qui règle chaque jour les seuils de maintenance et de stock, s'en approche à 1,1 % près, tandis que les agents qui décident machine par machine restent loin derrière. Un modèle de substitution (R^2 de 0,98 sur le coût) permet l'analyse de sensibilité de Sobol, et une application web restitue l'ensemble.

Sommaire

1 Introduction 3

2 Données et calibration 3

2.1 Le jeu de données Azure . . . 3

2.2 Lois de durée de vie . . . 4

2.3 Validation du simulateur . . . 4

3 Simulateur 5

4 Prédiction des pannes 6

4.1 Jeux d'apprentissage . . . 6

4.2 Résultats . . . 6

5 Politiques de référence 7

6 Apprentissage par renforcement 7

6.1 Formulation . . . 7

6.2 Trois niveaux d'action . . . 8

6.3 Algorithmes . . . 8

6.4 Courbes d'apprentissage 10

7 Modèle de substitution et sensibilité 11

8 Évaluation finale 12

8.1 Protocole 12

8.2 Scénario nominal 13

8.3 Robustesse 14

9 Application web 15

10 Conclusion 17

10.1 Bilan 17

10.2 Limites 18

10.3 Perspectives 19

A Reproduire les résultats 19

Références 20

1 Introduction

Ce rapport présente la réalisation du projet défini par le cahier des charges : un simulateur à événements discrets d'une flotte de 100 machines, des modèles d'apprentissage profond qui anticipent les pannes, des agents d'apprentissage par renforcement qui décident des commandes et des maintenances préventives, un modèle de substitution qui accélère l'évaluation de scénarios, et une application web qui restitue l'ensemble.

La question posée par le cahier des charges reste le fil conducteur : une politique pilotée par l'apprentissage profond réduit-elle le coût total de la flotte, à disponibilité au moins égale, par rapport à une politique à seuils fixes correctement réglée ? La section 8 y répond à partir de 30 années simulées par politique.

Le code, les configurations et les résultats bruts sont regroupés dans le dépôt du projet ; l'annexe A donne les commandes qui reproduisent chaque résultat.

2 Données et calibration

2.1 Le jeu de données Azure

Le jeu Microsoft Azure Predictive Maintenance [1] confirme les volumes annoncés dans le cahier des charges.

Tableau 1. Volumes du jeu de données Azure

Fichier	Lignes
Télémétrie horaire (tension, rotation, pression, vibration)	876 100
Erreurs non bloquantes	3 919
Remplacements de composants	3 286
Pannes	761
Machines	100

Les pannes se répartissent entre 192 pour comp1, 259 pour comp2, 131 pour comp3 et 179 pour comp4. Les machines appartiennent à quatre modèles (16, 17, 35 et 32 machines) et ont de 0 à 20 ans. Les moyennes et écarts-types des signaux (170,8 et 15,5 pour la tension, 446,6 et 52,7 pour la rotation, 100,9 et 11,0 pour la pression, 40,4 et 5,4 pour la vibration) correspondent aux niveaux utilisés par le générateur de télémétrie du simulateur.

L'analyse du journal de maintenance révèle une pratique particulière : tous les remplacements sont datés de 6 heures, et les remplacements planifiés ont lieu lors de tournées tous les 15 jours. L'intervalle médian entre deux remplacements planifiés d'un même composant est de 24 à 30 jours, avec un écart-type voisin de la moyenne (environ 50 jours).

2.2 Lois de durée de vie

Une durée de vie s'étend entre deux remplacements d'un même composant. Elle est complète lorsque le remplacement qui la termine fait suite à une panne, censurée à droite lorsqu'il s'agit d'un remplacement planifié ou de la fin de l'historique. Les lois de Weibull sont ajustées par maximum de vraisemblance avec la bibliothèque lifelines.

Tableau 2. Lois de Weibull ajustées sur Azure

Composant	Forme β	Échelle η	Pannes	Durées
comp1	1,80	4 234 h	183	802
comp2	1,50	3 669 h	256	861
comp3	1,90	5 093 h	128	806
comp4	1,95	4 332 h	176	810

Les quatre composants ont un paramètre de forme supérieur à 1 : leur taux de panne croît avec l'âge, et la maintenance préventive a donc un intérêt, ce que le cahier des charges demandait de vérifier (§ 3.2). Un modèle de Weibull à temps de vie accéléré précise l'effet des machines : chaque année d'âge réduit la durée de vie des composants d'environ 2 %, et les modèles 2, 3 et 4 vivent respectivement 17, 46 et 48 % plus longtemps que le modèle 1.

2.3 Validation du simulateur

Pour valider le simulateur (objectif O2), nous rejouons la politique observée dans Azure : chaque remplacement planifié du journal est appliqué à sa date, machine par machine, du 1er juin 2014 au 1er janvier 2016, sur 100 machines fonctionnant 24 heures sur 24. Les pannes ne proviennent alors que des lois de Weibull.

Avec les lois ajustées directement, le simulateur produit environ 30 % de pannes de moins qu'Azure, de façon homogène sur les quatre composants. L'hypothèse de censure indépendante est donc en défaut : les remplacements planifiés d'Azure visent probablement des composants déjà usés, ce qui rend la censure informative et fait surestimer les durées de vie. Nous corrigeons l'échelle de chaque loi par simulation, sur le premier semestre 2015 seulement : le nombre de pannes variant à peu près comme $\eta^{-\beta}$, l'échelle est multipliée à chaque itération par le rapport des pannes simulées et observées, élevé à la puissance $1/\beta$. Les échelles corrigées valent 0,72 à 0,81 fois les échelles ajustées.

Tableau 3. Écart entre pannes simulées et observées, moyenne sur 8 graines

Composant	Lois ajustées, 2e semestre	Lois corrigées, 2e semestre	Lois corrigées, année
comp1	−25,0 %	+18,2 %	+7,9 %
comp2	−30,6 %	−14,3 %	−4,3 %
comp3	−33,7 %	−2,0 %	−3,7 %
comp4	−29,6 %	+6,4 %	+6,4 %

Sur l'année 2015, l'écart reste inférieur à 10 % pour chaque composant, ce qui satisfait le critère O2 ; ce résultat inclut toutefois le semestre de calibration. Sur le seul second semestre, que la correction n'a pas vu, deux composants s'écartent de 14 et 18 %. Cet écart est du même ordre que la variabilité d'Azure elle-même : les pannes réelles de comp1 baissent de 22 % d'un semestre à l'autre, et l'incertitude de Poisson sur 84 pannes atteint déjà 11 %. Un critère de 10 % par semestre est donc proche du bruit statistique des données.

3 Simulateur

Le simulateur suit l'architecture décrite au § 5.1 du cahier des charges. Développé avec SimPy, il gère les pannes, interventions et livraisons comme des processus concurrents qui se partagent le stock et l'équipe de techniciens. Le vieillissement des 400 composants est suivi heure par heure de façon vectorisée. La production, les défauts, la télémétrie et les alarmes dépendent de l'usure, qui varie de moins de 1 % par jour : leurs espérances sont évaluées par blocs de six heures, et juste avant chaque remplacement, puis les quantités sont tirées une fois par jour. Cette organisation conserve la loi des quantités journalières tout en ramenant la simulation d'une année à 1,1 à 1,4 seconde, sous l'exigence de 2 secondes du cahier des charges.

Le simulateur enregistre pour chaque machine l'état hors service dominant de chaque journée et les heures d'arrêt pendant les heures d'ouverture, ce qui alimente les fiches machines de l'application. Il accepte des visites préventives groupées, au cours desquelles un technicien remplace plusieurs composants d'une même machine, comme le fait Azure. Une fenêtre glissante de 48 heures de télémétrie peut être activée pour les politiques prédictives ; elle est normalisée exactement comme les jeux d'apprentissage.

Deux scénarios sont fournis : S1, la flotte hétérogène à deux familles demandée par l'encadrement, et S0, une flotte de machines identiques servant à la validation. Les paramètres de départ sont ceux du chapitre 4 du cahier des charges. La règle de mise en tension y est respectée : la politique P0 connaît environ 16 % de ruptures et la politique optimisée P1 environ 1 %, si bien qu'un stock abondant n'est pas une solution évidente.

Le code est couvert par des tests automatiques : invariants du simulateur (stock jamais négatif, temps de marche inférieur au temps d'ouverture, reproductibilité à graine fixée), API, environnement d'apprentissage, modèles, et un parcours de bout en bout de l'application dans Microsoft Edge.

4 Prédiction des pannes

4.1 Jeux d'apprentissage

Chaque exemple décrit une machine au début d'une journée : une fenêtre de 48 heures des signaux, des attributs statiques et, pour chaque composant, une cible de survie en temps discret sur 14 jours, censurée par un remplacement préventif ou la fin de l'historique [2].

- **Données simulées** : cinq années de la flotte S1 sous la politique P0, soit 181 000 exemples ; trois années servent à l'apprentissage, une à la validation et une au test. Seules 0,13 % des machines-jours connaissent une panne dans les 24 heures.
- **Données réelles** : 36 300 exemples tirés d'Azure, avec le nombre d'erreurs à la place de l'indicateur de marche et le temps écoulé depuis le dernier remplacement de chaque composant. Le découpage est temporel, comme le prévoit le protocole : janvier à août pour l'apprentissage, septembre et octobre pour la validation, novembre et décembre pour le test.

Un remplacement daté de l'instant de prédiction trahit une panne enregistrée au même instant : le temps écoulé depuis le dernier remplacement ne compte donc que les remplacements strictement antérieurs. Un test automatique vérifie l'absence de cette fuite.

4.2 Résultats

Les seuils de décision sont choisis sur la période de validation, les métriques sont mesurées sur le test. L'aire sous la courbe précision-rappel (PR-AUC) se compare à la proportion de pannes : 0,13 % à 24 heures sur données simulées, 2,1 % sur Azure.

Tableau 4. Prédiction sur données simulées (année de test)

Modèle	F1, 24 h	PR-AUC, 24 h	F1, 7 jours	PR-AUC, 7 jours
LightGBM	0,090	0,030	0,424	0,310
LSTM	0,160	0,098	0,440	0,379
GRU	0,154	0,098	0,445	0,405
TCN	0,146	0,103	0,450	0,403

Tableau 5. Prédiction sur données réelles Azure (novembre et décembre 2015)

Modèle	F1, 24 h	PR-AUC, 24 h	F1, 7 jours	PR-AUC, 7 jours
LightGBM	0,844	0,925	0,268	0,314
LSTM	0,422	0,337	0,350	0,299
GRU	0,573	0,561	0,361	0,324
TCN	0,646	0,655	0,370	0,325

Sur données simulées, les réseaux de survie dépassent LightGBM aux deux horizons, le TCN et le GRU obtenant le meilleur classement à 7 jours. Sur Azure, LightGBM l'emporte nettement à 24 heures : les pannes de ce jeu synthétique sont précédées d'anomalies franches dans les dernières heures, que des moyennes glissantes sur 3 et 24 heures captent très bien. À 7 jours, horizon qui compte pour la gestion du stock, les réseaux reprennent l'avantage en F1 (0,37 contre 0,27). Le critère O3, qui demandait de dépasser LightGBM, est donc atteint sur données simulées et à 7 jours sur données réelles, mais pas à 24 heures sur données réelles.

Le premier entraînement de LightGBM donnait un résultat dégénéré, voisin du hasard : avec moins de 200 pannes par composant et un poids des pannes proche de 1 000, le modèle apprenait les exemples par cœur. Des arbres moins profonds, un poids plafonné et un sous-échantillonnage ont rétabli une référence équitable.

5 Politiques de référence

- **P0**, seuils fixes réglés à la main : stock en (s, S) selon le scénario, maintenance systématique aux intervalles constructeur, une pièce de chaque référence réservée aux pannes.
- **P1**, seuils fixes optimisés : recherche aléatoire de 40 combinaisons des seuils (s, S) et des facteurs d'intervalle de maintenance, évaluées sur les mêmes graines. Le coût annuel passe de 57 600 à 53 700 UR, les ruptures de 16 à 1 %.
- **P2**, maintenance conditionnelle : remplacement lorsque le signal associé au composant dérive, lorsque le taux de défauts dépasse un seuil, ou à 1,5 fois l'intervalle.
- **P3**, politique prédictive sans apprentissage par renforcement : le réseau de survie qui classe le mieux les pannes à 7 jours estime le risque de chaque composant. Un composant est remplacé lorsque son risque de panne à 3 jours dépasse un seuil ou lorsqu'il atteint un multiple donné de son intervalle de maintenance, et le stock est recomplété jusqu'à la demande prévue sur le délai fournisseur, augmentée d'un stock de sécurité. Sans la seconde règle, P3 laissait vieillir les composants : le taux de défauts passait de 2,4 à 3,1 % et le rebut coûtait 10 000 UR de plus par an. Ses trois paramètres sont réglés sur une grille de 18 combinaisons, avec les graines de P1 : la grille retient un seuil de risque de 5 %, un remplacement d'office à 1,5 fois l'intervalle et une marge de stock réduite à une pièce, pour un coût de 52 900 UR sur ces graines.
- **Oracle** : remplace chaque composant à 70 % de sa durée de vie réelle et commande les pièces dont il sait qu'elles serviront ; c'est une borne qu'aucune politique réaliste ne peut atteindre.

6 Apprentissage par renforcement

6.1 Formulation

L'agent décide une fois par jour, au début de la journée, à partir de l'état observable de la flotte. Un épisode couvre une année, soit 365 décisions, et chaque épisode tire une nouvelle graine du simulateur : l'agent ne revoit jamais deux fois la même suite de pannes. La récompense d'une

journée est l'opposé de son coût total, qui regroupe les huit postes du cahier des charges (stockage, achats, urgences, main-d'œuvre, surcoût correctif, arrêts, rebut et retouche), divisé par 150 pour ramener les récompenses vers l'unité. Maximiser la somme des récompenses revient donc exactement à minimiser le coût annuel, sans pondération arbitraire entre coût et disponibilité : les arrêts sont valorisés par la marge perdue.

L'observation ne contient que des grandeurs connues d'un responsable de maintenance réel : stock, commandes en cours et machines en attente pour chaque référence, avancement de chaque composant par rapport à son intervalle de maintenance (moyenne, maximum et part des composants au-delà de 80 et 100 %), part des machines en service, en attente de pièce ou de technicien, techniciens libres, taux de défauts et alarmes des dernières 24 heures, avancement de l'année. Les durées de vie réelles des composants restent cachées. Toutes les grandeurs sont normalisées par des valeurs de référence du scénario, ce qui donne 39 variables.

6.2 Trois niveaux d'action

Le cahier des charges prévoyait deux niveaux d'action (§ 5.3). Un troisième, plus simple, a été ajouté pour pouvoir appliquer un DQN classique, qui ne sait traiter qu'un petit nombre d'actions discrètes.

Tableau 6. Niveaux d'action proposés aux agents

Niveau	Décision quotidienne	Taille de l'espace
Réduit	Un seuil de maintenance commun à tous les composants et un multiplicateur de stock commun à toutes les références, chacun parmi six valeurs	36 actions
N1	Un seuil de maintenance par composant (0,6 à 1,5 fois l'intervalle, ou jamais) et un niveau de recomplètement par référence (0,5 à 3 fois le niveau de référence)	$6^8 \approx 1{,}7$ million de combinaisons
N2	Quantité commandée et commande urgente pour chaque référence, et pour chaque machine : ne rien faire ou remplacer l'un de ses quatre composants	plus de 10^{76} combinaisons

Au niveau N1, l'agent règle chaque jour les paramètres d'une politique à seuils, que le simulateur applique ensuite. Ce niveau reste lisible pour un responsable de maintenance : il voit à tout moment quels seuils l'agent a retenus. Au niveau N2, l'agent prend lui-même chaque décision. Les actions impossibles ou absurdes y sont masquées : maintenance d'une machine déjà arrêtée, d'un composant dont la pièce manque ou qui n'a pas atteint la moitié de son intervalle, ou alors qu'aucun technicien n'est libre. Pour ce niveau, l'observation est complétée, pour chaque machine, par son état et l'avancement de ses quatre composants, soit 539 variables.

6.3 Algorithmes

MaskablePPO pour P4 et P5

PPO [4] est un algorithme acteur-critique robuste aux espaces d'actions discrets multiples. Sa variante MaskablePPO de la bibliothèque sb3-contrib [9] exclut les actions invalides de la distribution de l'acteur, ce qui accélère nettement l'apprentissage par rapport à une pénalité [5]. Les deux agents partagent leurs réglages : trois environnements en parallèle, 365 pas par environnement entre deux mises à jour, lots de 730 transitions, taux d'apprentissage 3×10^{-4}, facteur d'actualisation 0,995 (horizon effectif de 200 jours, du même ordre que les délais et les durées de vie). P4 dispose de 300 000 pas, soit environ 820 années simulées, et d'un coefficient d'entropie de 0,01 ; P5 de 500 000 pas en deux temps, 200 000 puis 300 000 à partir du meilleur modèle du premier passage, sans bonus d'entropie. Ces choix, ainsi que la normalisation des récompenses et la conservation du meilleur modèle, sont justifiés plus bas. Au niveau N2, la loi masquée de sb3-contrib traite les 108 dimensions de l'action une à une et coûtait 77 ms par décision, quinze fois le temps de simulation d'une journée ; une version vectorisée, dont un test vérifie qu'elle donne les mêmes probabilités, ramène ce temps à 1,5 ms.

DQN au niveau réduit

Le DQN [3] de Stable-Baselines3 sert de référence : réseau de deux couches de 64 neurones, mémoire de 200 000 transitions, exploration ε décroissante sur les 10 premiers pourcents de l'apprentissage, 100 000 pas.

Branching Dueling Q-Network

Un DQN classique estime une valeur par action ; au niveau N2, il devrait en estimer plus de 10^{76}. L'architecture à branches de Tavakoli et al. [6] contourne cette explosion : un tronc commun de deux couches de 256 neurones alimente une valeur d'état et, pour chaque dimension de l'action, une tête d'avantages. Le nombre de sorties croît alors avec la somme des tailles des dimensions (près de 600) et non avec leur produit. La cible d'apprentissage suit le double DQN, en moyenne sur les branches, et les actions invalides sont masquées dans chaque branche. Ce réseau, absent des bibliothèques courantes, a été écrit directement avec PyTorch ; les sorties de toutes les branches sont rangées dans un même tableau pour éviter une boucle sur la centaine de branches. L'apprentissage dure 120 000 pas, avec une mise à jour tous les quatre pas sur des lots de 64 transitions, une mémoire de 100 000 transitions, un taux d'apprentissage de 10^{-4} et une exploration ε qui décroît de 1 à 0,05 sur les 30 premiers pourcents.

Point de départ des agents N2

Un agent non entraîné choisit presque uniformément entre les options de chaque dimension : au niveau N2, il commande chaque jour des quantités aléatoires et envoie en maintenance quatre machines sur cinq. Ces premières années coûtent plus de 600 000 UR, dix fois le coût normal, et la multitude d'interventions ralentit la simulation au point de ramener l'apprentissage à une vingtaine de pas par seconde. Nous ajoutons donc, avant apprentissage, un biais aux sorties des deux agents N2 qui rend l'option « ne rien faire » largement majoritaire ; l'exploration du BDQ tire ses actions aléatoires selon ce même biais. L'agent part ainsi d'une politique prudente et apprend quand s'en écarter, ce qui ne restreint aucune décision. Un premier apprentissage de P5 avec le bonus d'entropie habituel s'est d'abord amélioré, jusqu'à 86 000 UR par an après 14 000 pas, puis dégradé jusqu'à 106 000 UR après 54 000 pas : sommé sur 108 dimensions, ce bonus pousse l'acteur vers des choix uniformes, c'est-à-dire précisément vers les années absurdes que le biais initial écarte. Il a donc été supprimé au niveau N2.

Normalisation des récompenses

Sans le bonus d'entropie, la même dérive de P5 est apparue plus lentement. Le premier apprentissage de P4, mené sans précaution particulière, montrait un symptôme voisin : le coût moyen des épisodes est passé de 53 800 à 53 400 UR en 300 000 pas, autant dire qu'il n'a pas bougé. Dans les deux cas, la variance des retours expliquée par le critique restait nulle, si bien que les avantages estimés n'étaient guère que du bruit. Les retours, de l'ordre de -200, varient beaucoup d'une année à l'autre ; les récompenses sont donc divisées par l'écart-type courant des retours, ce qui ramène les cibles du critique vers l'unité. La variance expliquée est alors passée à plus de 0,5 dès les premières mises à jour. Les deux agents ont été réentraînés ainsi, et le modèle conservé est celui dont la moyenne des derniers épisodes d'apprentissage est la meilleure, et non le dernier.

6.4 Courbes d'apprentissage

La figure 1 suit le coût annuel moyen des derniers épisodes au fil de l'apprentissage. Ces épisodes sont joués avec la politique stochastique de l'agent, ou avec son exploration pour les deux DQN, et sur des graines chaque fois nouvelles : la courbe mesure la progression, pas la performance finale, que la section 8 évalue sur des années communes à toutes les politiques.

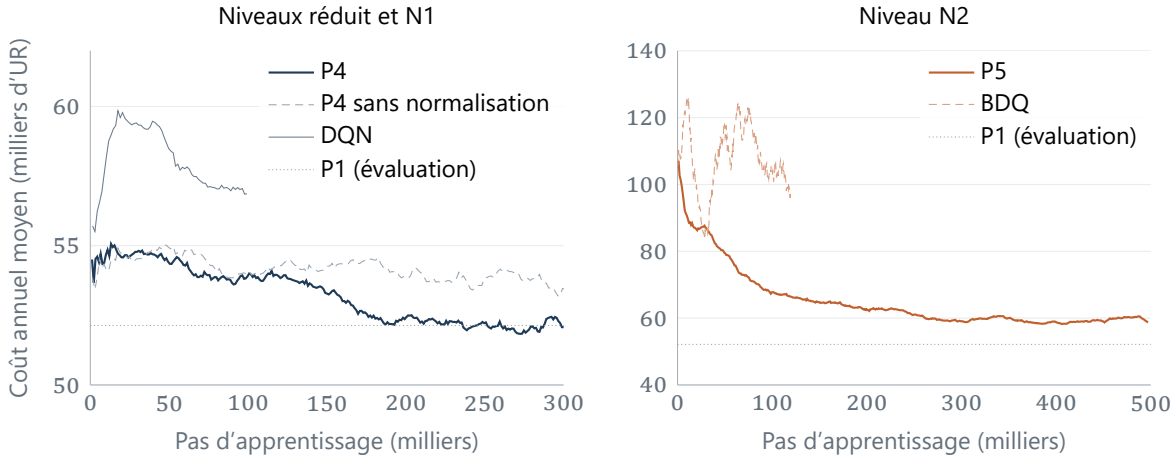


Figure 1. Coût annuel moyen des épisodes d'apprentissage : moyenne glissante des 100 derniers épisodes pour PPO et DQN, des 20 derniers pour le BDQ. Le pointillé rappelle le coût de P1 mesuré lors de l'évaluation finale.

Une fois les récompenses normalisées, les deux agents PPO progressent nettement. P4 passe de 54 500 à 51 800 UR sur ses épisodes d'apprentissage, quand sa première version restait autour de 53 500 UR. P5 part de 107 000 UR et descend à 62 500 UR en 200 000 pas. Comme sa courbe baissait encore, l'apprentissage a été prolongé de 300 000 pas à partir de ce modèle : P5 atteint 58 200 UR, puis plafonne autour de 59 000 UR sur les 100 000 derniers pas. La durée d'apprentissage n'est donc plus ce qui le limite. Le BDQ progresse tant qu'il explore, jusqu'à 83 500 UR, puis se dégrade lorsque l'exploration s'éteint : les valeurs de ses 108 branches sont apprises sur une récompense commune à toute la flotte, et l'effet d'une décision isolée, comme l'entretien d'une machine, se perd dans le bruit des pannes des 99 autres. Le DQN, enfin, n'améliore jamais la politique aléatoire dont il part, autour de 56 000 UR : un seuil commun à tous les composants et un niveau de stock commun à toutes les références ne laissent guère de marge.

Jouer l'action la plus probable ou tirer les actions

Une politique PPO est une loi de probabilité sur les actions. L'usage veut qu'on l'évalue en jouant son mode, l'action la plus probable de chaque dimension, mais ce mode peut s'écarter beaucoup de la politique apprise. Au niveau N2, « ne rien commander » reste l'option la plus probable de chaque référence : l'agent apprend à commander certains jours seulement, et son mode ne commande jamais. Joué ainsi, P5 coûte 432 500 UR par an sur les graines de réglage, contre 61 800 UR en tirant ses actions. Pour chaque agent PPO, le mode de jeu est donc choisi sur les graines de réglage de P1 et P3, jamais sur celles de l'évaluation finale, et le tirage est initialisé par la graine de la simulation pour rester reproductible. Le tirage l'emporte pour les deux agents, nettement pour P5 et de 1 700 UR pour P4.

7 Modèle de substitution et sensibilité

Le modèle de substitution est un ensemble de cinq réseaux [7] entraîné sur 2 000 scénarios tirés en hypercube latin et simulés deux fois chacun sur 180 jours, soit 24 minutes de calcul. Ses entrées sont le nombre de techniciens, les facteurs de délai fournisseur et de taux de panne, et les facteurs d'intervalle de maintenance et de niveau de stock d'une politique à seuils fixes.

Tableau 7. Qualité du modèle de substitution sur 20 % de scénarios non vus

Indicateur	R^2	Erreur relative moyenne
Coût total	0,982	2,1 %
TRS	0,978	0,3 %
Disponibilité	0,975	0,3 %
Fréquence de ruptures	0,983	non pertinente
Attente de pièce	0,913	non pertinente
Pannes	0,983	3,2 %

Le critère O5 (R^2 supérieur à 0,9) est atteint pour chaque indicateur. L'erreur relative n'a pas de sens pour les ruptures et l'attente de pièce, souvent proches de zéro. Le modèle répond en quelques millisecondes, contre environ une seconde par scénario simulé : la page « Scénarios » de l'application l'interroge à chaque mouvement de curseur.

Il permet aussi l'analyse de sensibilité globale prévue au § 5.4, impossible à mener directement sur le simulateur : 14 336 évaluations selon l'échantillonnage de Saltelli [8] donnent les indices de Sobol totaux de la figure 2.

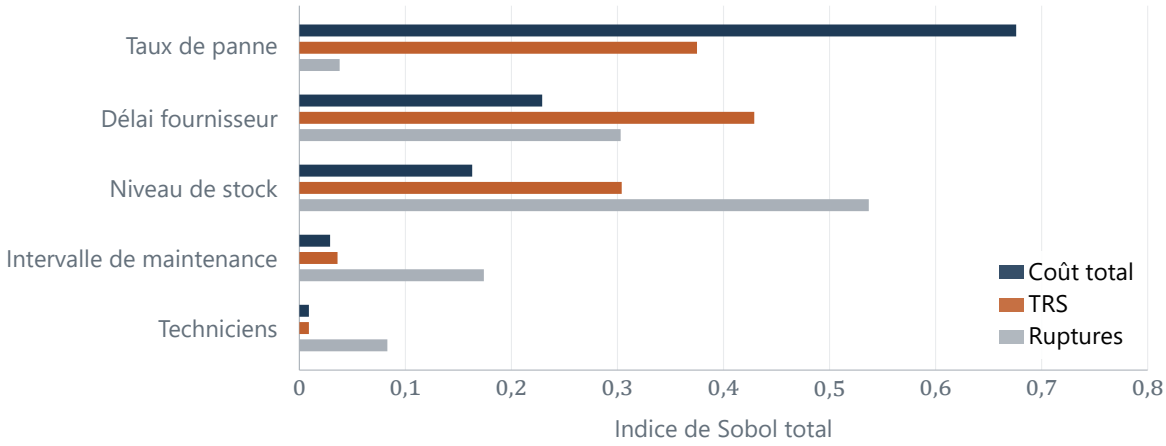


FIGURE 2. Sensibilité des indicateurs aux paramètres du système et de la politique.

Le taux de panne explique l'essentiel de la variance du coût. Le délai fournisseur et le niveau de stock dominent le TRS et les ruptures. Le nombre de techniciens n'a presque aucun effet, cohérent avec leur charge moyenne de 27 % : l'équipe n'est pas le goulot d'étranglement du scénario nominal.

8 Évaluation finale

8.1 Protocole

Le protocole est celui du § 7.3 du cahier des charges. Chaque politique est simulée sur les mêmes 30 années, de graines 9000 à 9029, qu'aucun réglage ni apprentissage n'a utilisées. Les nombres aléatoires communs font que deux politiques affrontent exactement les mêmes durées de vie et les mêmes aléas de délai : l'écart observé sur une graine ne doit rien au hasard du tirage. Les moyennes sont accompagnées de leur intervalle de confiance à 95 %, et l'écart de coût de chaque politique à P1 est testé par un test de Wilcoxon apparié sur les 30 années.

8.2 Scénario nominal

Tableau 8. Indicateurs moyens sur 30 années, scénario S1 nominal

Pol.	Coût (UR)	IC 95 %	TRS	Dispo.	Ruptures	Attente (h)	p
P0	55 355	53 702 à 57 008	96,08 %	98,43 %	12,4 %	33,1	$< 0{,}001$
P1	52 136	50 882 à 53 391	96,69 %	99,06 %	1,1 %	2,1	réf.
P2	47 135	45 574 à 48 697	96,71 %	98,85 %	23,9 %	110,1	$< 0{,}001$
P3	51 038	49 936 à 52 139	96,70 %	99,25 %	2,6 %	3,8	0,004
DQN	58 483	56 807 à 60 159	95,65 %	98,36 %	21,8 %	33,6	$< 0{,}001$
P4	52 725	51 333 à 54 117	96,61 %	98,94 %	2,4 %	8,9	0,029
P5	59 644	57 610 à 61 678	95,72 %	98,23 %	6,0 %	23,3	$< 0{,}001$
BDQ	92 844	89 493 à 96 195	94,00 %	97,24 %	3,8 %	42,6	$< 0{,}001$
Oracle	42 408	41 755 à 43 061	97,29 %	99,28 %	10,2 %	35,5	$< 0{,}001$

La colonne p donne la probabilité, sous l'hypothèse d'une absence d'écart avec P1, d'observer un écart de coût au moins aussi marqué. Les ruptures sont la part des demandes de pièces après panne que le stock n'a pas pu servir immédiatement ; l'attente est la durée moyenne d'attente de pièce par réparation, en heures.

La prédiction réduit le coût, la surveillance d'état davantage encore

P3, qui s'appuie sur le réseau de survie, coûte 51 000 UR par an, 2,1 % de moins que P1, et l'écart est significatif ($p = 0{,}004$). Il ramène les pannes de 189 à 48 par an et porte la disponibilité à 99,25 %, la meilleure des politiques réalistes. La maintenance conditionnelle P2 fait pourtant mieux, avec 47 100 UR, soit 9,6 % de moins que P1, sans aucun réglage. Elle ramène les pannes à 30 par an et, avec si peu de pannes, se contente des niveaux de stock de P0 : la valeur moyenne du stock est de 208 UR, contre 1 065 pour P3 et 1 105 pour P1. Elle surveille aussi le taux de défauts, que le réseau de survie ignore : P3 laisse les composants s'user un peu plus, avec 2,6 % de pièces défectueuses contre 2,2 % pour P2. L'avantage de P2 tient donc moins à la qualité de sa détection qu'à ce qu'elle observe : la dérive du signal propre à chaque composant, que le simulateur relie directement à l'usure, est une information plus directe qu'un risque de panne estimé.

La fréquence de ruptures se lit avec le nombre de pannes

P2 présente 23,9 % de ruptures, mais sur 30 demandes seulement : cela représente 7,5 ruptures par an, contre 2,1 pour P1. Ces attentes, longues (110 heures en moyenne) mais rares, ne pèsent presque pas sur le TRS, identique à celui de P1 et de P3 (96,7 %).

Les agents d'apprentissage par renforcement n'atteignent pas P1

P4 est l'agent le plus proche : 52 700 UR par an, 1,1 % de plus que P1, écart faible mais significatif ($p = 0{,}03$), pour un TRS presque identique (96,61 contre 96,69 %). Il connaît autant de pannes que P1 (192 contre 189 par an) avec un stock plus léger (851 UR contre 1 105) mais plus de

ruptures (4,7 contre 2,1 par an). Ce résultat tient pour une large part à son espace d'action : au niveau N1, l'agent choisit chaque jour des seuils d'âge et des niveaux de stock, c'est-à-dire une politique de la même famille que P1. Il peut adapter ces seuils à l'état du magasin, mais il ne peut pas déclencher une maintenance sur l'état d'un composant, ce qui fait l'avantage de P2 et de P3. Le gain possible se limite donc à ce qu'apporte l'adaptation des seuils, et cet agent ne l'a pas encore trouvé.

Au niveau N2, où cette limite disparaît, les agents restent loin du compte : P5 coûte 59 600 UR, 14 % de plus que P1, et le BDQ 92 800 UR. P5 n'a pas appris à cibler la maintenance préventive : avec presque autant d'interventions préventives que P1 (600 contre 640 par an), il laisse survenir 249 pannes, contre 189. Il remplace des composants, mais rarement ceux qui allaient tomber en panne. Le BDQ a presque renoncé à la maintenance préventive (27 interventions par an, 391 pannes) et accumule les commandes. Le DQN du niveau réduit, enfin, fait moins bien que P0. Avec le budget de calcul de ce projet, l'apprentissage par renforcement ne retrouve donc pas ce que les règles conçues à la main obtiennent ; la section 10 y revient.

Ce qui reste à gagner

L'oracle, qui connaît la durée de vie réelle de chaque composant, coûte 42 400 UR par an. L'écart de 10 % avec P2 mesure la valeur d'une information parfaite sur l'état des composants ; il borne ce qu'un meilleur prédicteur pourrait encore apporter.

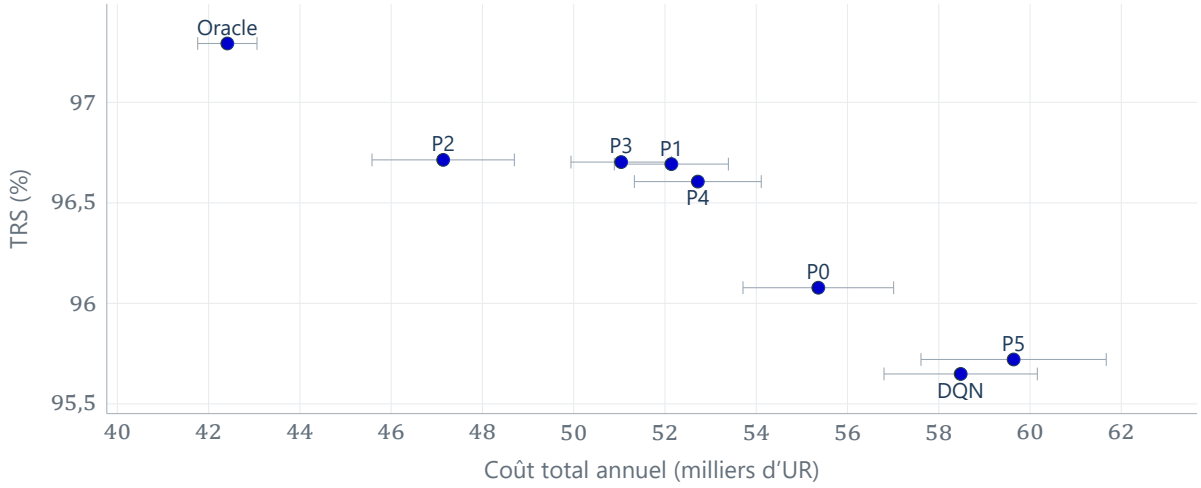


Figure 3. Coût et TRS moyens de chaque politique ; les barres donnent l'intervalle de confiance du coût. Les politiques les plus favorables se trouvent en haut à gauche : coût faible, TRS élevé. Le BDQ (92 800 UR, TRS de 94,0 %) sort du cadre.

8.3 Robustesse

Les politiques réglées ou apprises sur le scénario nominal sont ensuite confrontées, sans nouveau réglage, à trois situations dégradées : délais fournisseurs doublés, taux de panne augmenté de 30 %, équipe réduite à un technicien. Le tableau 9 donne le coût moyen et sa variation par rapport au scénario nominal.

Tableau 9. Coût moyen sur 30 années en situation dégradée (milliers d'UR)

Pol.	Nominal	Délais doublés	Pannes +30 %	Un technicien
P0	55,4	69,5 (+26 %)	62,7 (+13 %)	59,4 (+7 %)
P1	52,1	55,3 (+6 %)	57,6 (+11 %)	57,3 (+10 %)
P2	47,1	58,7 (+25 %)	55,0 (+17 %)	57,7 (+22 %)
P3	51,0	54,4 (+6 %)	57,1 (+12 %)	59,0 (+16 %)
DQN	58,5	79,6 (+36 %)	66,9 (+14 %)	63,3 (+8 %)
P4	52,7	57,0 (+8 %)	58,9 (+12 %)	58,4 (+11 %)
P5	59,6	63,2 (+6 %)	66,5 (+11 %)	62,0 (+4 %)
BDQ	92,8	104,9 (+13 %)	96,5 (+4 %)	97,8 (+5 %)
Oracle	42,4	45,5 (+7 %)	46,4 (+9 %)	50,9 (+20 %)

Délais fournisseurs doublés

C'est la situation qui sépare le plus les politiques. P1 et P3 ne voient leur coût croître que de 6 % : P1 dispose de stocks élevés, et P3 dimensionne ses commandes sur la demande prévue pendant le délai, qui s'allonge avec lui. P2, dont le stock est bas, prend 25 % et passe derrière elles. P3 devient la politique réaliste la moins chère, à 54 400 UR, sans que l'écart avec P1 soit significatif ($p = 0{,}34$).

Taux de panne augmenté de 30 %

L'ordre du scénario nominal se maintient : P2 reste la moins chère, à 55 000 UR, P3 et P1 restent proches l'une de l'autre ($p = 0{,}12$).

Un seul technicien

Les politiques qui multiplient les interventions préventives souffrent le plus du manque de main-d'œuvre : P2 prend 22 %, P3 16 % et l'oracle lui-même 20 %. P1 devient la politique réaliste la moins chère, à égalité statistique avec P2 ($p = 0{,}19$). Une politique fondée sur l'état des composants suppose donc une équipe capable d'absorber les interventions qu'elle déclenche ; les politiques étudiées se bornent à ne pas lancer plus d'interventions qu'il n'y a de techniciens libres le matin, sans planifier leurs remplacements pour lisser la charge.

9 Application web

L'application se compose d'une interface React et d'une API FastAPI qui expose le moteur Python. Les outils de tableau de bord en Python ont été écartés au profit d'une interface conçue comme un produit : son système visuel (typographie Archivo et IBM Plex, palette violette, jaune et blanche, thème clair) est consigné dans un guide de style et suit le catalogue d'anti-modèles d'impeccable.style [10]. Elle compte sept pages.

- **Tableau de bord** : bilan d'une année simulée, évolution du TRS, coûts cumulés par poste,

machines à l'arrêt.

- **Flotte en direct** : la simulation est diffusée jour après jour par WebSocket ; chaque machine est une case colorée selon son état, avec l'état du magasin, la file des techniciens et un journal d'événements rédigé en français.
- **Machines** et fiches machines : frise des états, télémétrie, taux de défauts, historique.
- **Stock** : niveau, commandes en cours et machines en attente pour chaque référence, avec les seuils de la politique.
- **Comparaison** : indicateurs de plusieurs politiques avec intervalles de confiance, nuage coût et TRS.
- **Scénarios** : réponse instantanée du modèle de substitution, vérifiable par simulation.
- **Données** : jeu Azure, lois de durée de vie, validation et qualité des modèles de prédiction.

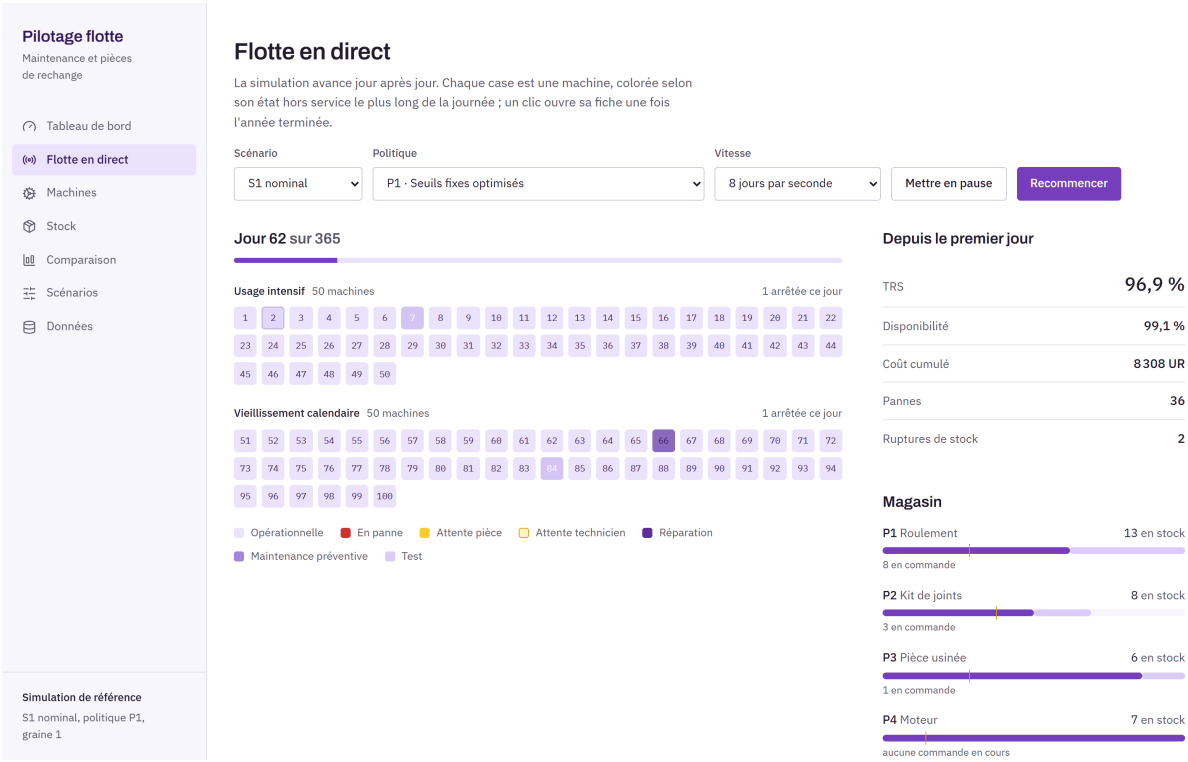


FIGURE 4. Page « Flotte en direct » en cours de simulation.

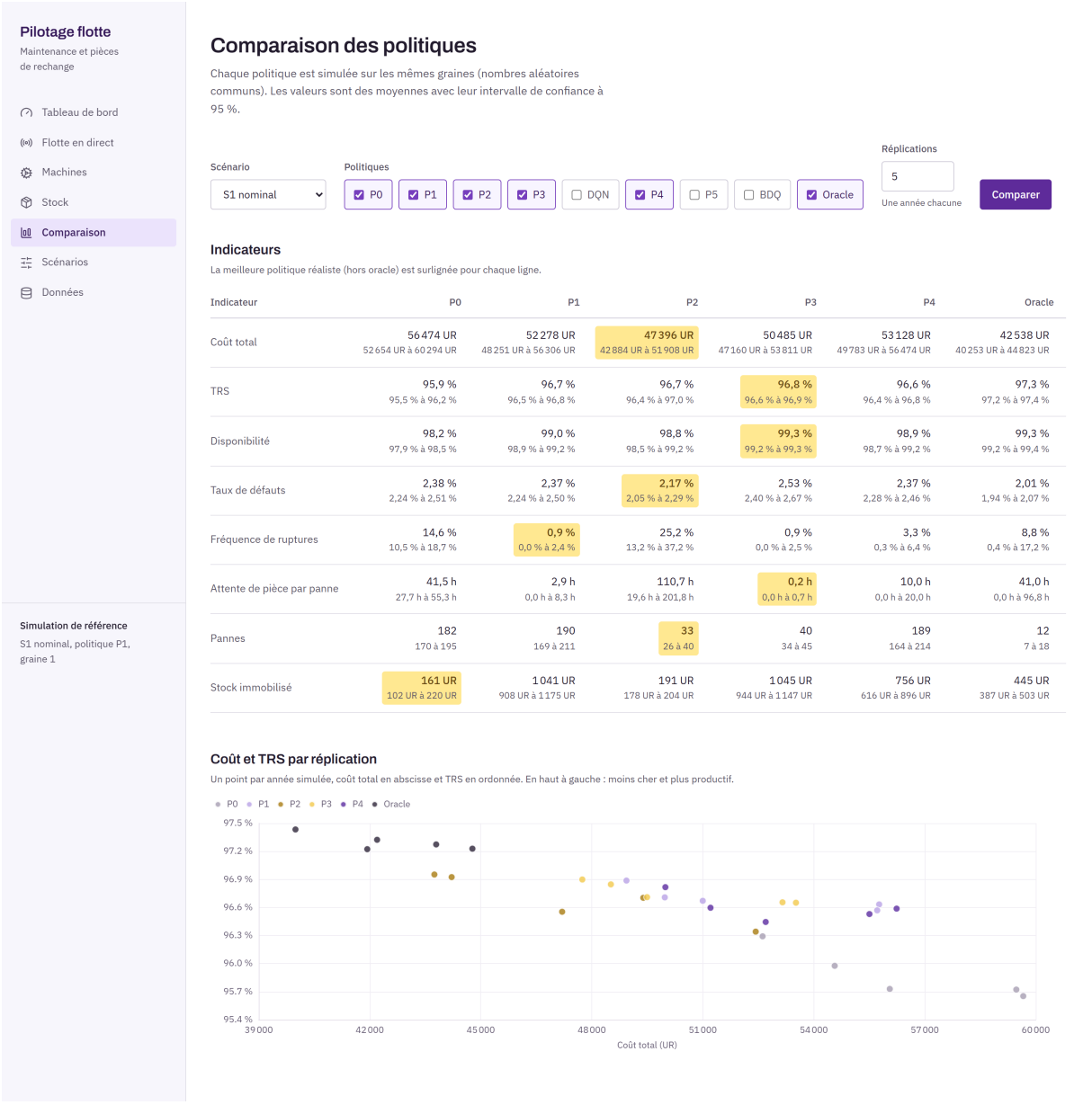


FIGURE 5. Page « Comparaison » : indicateurs avec intervalles de confiance et nuage coût, TRS.

10 Conclusion

10.1 Bilan

La question posée par le cahier des charges appelle une réponse nuancée. L'apprentissage profond réduit bien le coût par rapport à une politique à seuils fixes correctement réglée, mais par la prédiction plutôt que par l'apprentissage par renforcement : P3, qui s'appuie sur un réseau de survie, économise 2,1 % à disponibilité supérieure, et c'est la politique la plus robuste lorsque les délais fournisseurs doublent. Les agents d'apprentissage par renforcement, eux, n'atteignent pas P1 ; le meilleur s'en approche à 1,1 % près. Une règle de maintenance conditionnelle simple fait mieux que toutes les politiques apprises, avec 9,6 % d'économie.

Deux enseignements en ressortent. Le premier est que la valeur vient de l'information sur l'état des composants : les politiques qui l'exploitent, P2 et P3, divisent le nombre de pannes par quatre à six, et l'oracle montre qu'il reste encore 10 % à gagner avec une information parfaite. Le second est que l'apprentissage par renforcement bute ici sur des difficultés concrètes, que le projet a rencontrées et levées une à une : un espace d'actions de plus de 10^{76} combinaisons, une récompense bruitée par les pannes de toute la flotte, un critique incapable d'apprendre sans normalisation des récompenses, une politique dont l'action la plus probable diffère de la loi apprise. Chacune est documentée avec sa correction, ce qui vaut autant pour la suite que les chiffres eux-mêmes.

Tableau 10. Atteinte des objectifs du cahier des charges

Obj.	Critère et résultat	Verdict
O1	Jeu Azure enrichi (durées, familles, défauts, coûts) ; une fuite d'information temporelle détectée, corrigée et couverte par un test	Atteint
O2	Pannes d'Azure reproduites à 10 % près : écarts de −4,3 à +7,9 % sur 2015, jusqu'à 18 % sur le seul second semestre	Atteint sur l'année
O3	F1 supérieur à LightGBM : vrai aux deux horizons sur données simulées, à 7 jours seulement sur Azure	Atteint en partie
O4	Coût inférieur à P1 à disponibilité au moins égale : P4 à +1,1 %, P5 à +14 %	Non atteint
O5	R^2 supérieur à 0,9 sur des scénarios non vus : de 0,913 à 0,983	Atteint
O6	Application web de sept pages, parcours principal testé de bout en bout	Atteint

10.2 Limites

- **Des données réelles elles-mêmes synthétiques.** Le jeu Azure a été généré par Microsoft pour illustrer la maintenance prédictive. Ses pannes sont précédées d'anomalies plus nettes qu'on ne l'observe souvent sur le terrain, ce qui explique en partie le bon score de LightGBM à 24 heures, et sa calibration repose sur une censure informative que nous avons dû corriger par simulation.
- **Un simulateur volontairement simplifié.** Les coûts sont exprimés en unités relatives, les fournisseurs n'ont pas de capacité limitée, une pièce déposée n'est jamais réparée ni réutilisée, et les techniciens sont interchangeables.
- **Un budget de calcul limité.** Chaque agent a été entraîné avec une seule graine, en quelques centaines de milliers de pas, sur un poste de quatre cœurs, avec des réglages proches des valeurs usuelles. Les écarts entre agents mesurent donc ces entraînements précis plus que les méthodes elles-mêmes ; plusieurs graines d'entraînement, des apprentissages plus longs et un réglage des hyperparamètres seraient nécessaires pour conclure sur les méthodes.
- **Un seul scénario d'apprentissage.** Les agents et les seuils sont réglés sur le scénario nominal ; la robustesse est mesurée sans nouvel apprentissage, ce qui correspond à l'usage réel mais

pénalise les politiques les plus spécialisées.

10.3 Perspectives

- Donner aux agents du niveau N2 une récompense par machine, qui attribuerait à chaque branche l'effet de sa propre décision : prolongé jusqu'à 500 000 pas, P5 plafonne autour de 59 000 UR, signe que la durée d'apprentissage n'est plus le facteur limitant.
- Donner aux agents le risque estimé par le réseau de survie : le codec d'observation le prévoit déjà, et l'agent combinerait alors l'information de P3 et l'adaptation de l'apprentissage par renforcement.
- Entraîner les agents sur un ensemble de scénarios tirés au hasard (délais, taux de panne, effectifs) plutôt que sur le seul scénario nominal, pour obtenir des politiques robustes par construction.
- Remplacer la recherche aléatoire de P1 et la grille de P3 par une optimisation bayésienne sur le modèle de substitution, qui évalue un jeu de paramètres en quelques millisecondes.
- Confronter le simulateur et les modèles à l'historique d'un atelier réel, qui fournirait les durées, les coûts et les pratiques de maintenance que le jeu Azure ne contient pas.

A Reproduire les résultats

Prérequis : Python 3.11 avec uv, Node.js, les cinq fichiers CSV d'Azure dans `data/raw`.

Commande	Résultat
`uv sync --extra ml`	Installation des dépendances
`uv run pytest`	Tests automatiques
`scripts/calibrate_azure.py`	Lois de Weibull (tableau 2)
`scripts/validate_azure.py`	Validation O2 (tableau 3)
`scripts/optimize_p1.py`	Seuils de P1
`scripts/train_predictor.py`	Tableau 4 ; avec `--source azure`, tableau 5
`scripts/tune_p3.py`	Seuils de P3
`scripts/train_rl.py --level n1`	Agent P4 ; `--level n2` pour P5, prolongé avec `--resume`
`scripts/train_bdq.py`	Branching Dueling Q-Network
`scripts/train_rl.py --level reduced --algo dqn`	DQN du niveau réduit
`scripts/choose_agent_mode.py`	Mode de jeu de P4 et P5
`scripts/train_surrogate.py`	Modèle de substitution
`scripts/sensitivity.py`	Indices de Sobol (figure 2)

`scripts/evaluate_policies.py --robustness`	Évaluation finale (section 8)
`scripts/export_report_data.py`	Données des graphiques de ce rapport

Références

[1] Microsoft Azure Predictive Maintenance, jeu de données publié sur Kaggle, https://www.kaggle.com/datasets/arnabbiswas1/microsoft-azure-predictive-maintenance.

[2] M. F. Gensheimer et B. Narasimhan, « A scalable discrete-time survival model for neural networks », *PeerJ*, 7:e6257, 2019.

[3] V. Mnih et al., « Human-level control through deep reinforcement learning », *Nature*, 518, p. 529-533, 2015.

[4] J. Schulman et al., « Proximal policy optimization algorithms », arXiv:1707.06347, 2017.

[5] S. Huang et S. Ontañón, « A closer look at invalid action masking in policy gradient algorithms », *FLAIRS*, 2022.

[6] A. Tavakoli, F. Pardo et P. Kormushev, « Action branching architectures for deep reinforcement learning », *AAAI*, 2018.

[7] B. Lakshminarayanan, A. Pritzel et C. Blundell, « Simple and scalable predictive uncertainty estimation using deep ensembles », *NeurIPS*, 2017.

[8] A. Saltelli et al., *Global Sensitivity Analysis: The Primer*, Wiley, 2008.

[9] A. Raffin et al., « Stable-Baselines3: Reliable reinforcement learning implementations », *Journal of Machine Learning Research*, 22(268), 2021.

[10] Impeccable, catalogue d'anti-modèles d'interface, https://impeccable.style.